(continued)

```
// .env
OPENAI_API_KEY=<OPEN_AI_SECRET_KEY>
GEMINI_API_KEY=<GOOGLE_AI_SECRET_KEY>
```

The following listing shows how we implemented this in the example project.

Listing 8.2 Integrating error tracking into the application

```
import { AIErrorTracker } from '@/lib/error-tracking';
export async function continueConversation(input, provider, model) {
   ...
} catch (error) {
    const errorData =
await AIErrorTracker.trackError(error, {
      provider,
      model,
      input,
    });

    const userError =
AIErrorTracker.createUserFacingError(errorData);
    return {
      id: userError.requestId,
      role: 'assistant',
      display: (
        <ChatBubble
          role="error"
          text={`${userError.message} (Request ID: ${userError.requestId})`}
          className="mr-auto border-none error"
        />
      ),
    };
  }
```

Imports the custom AIErrorTracker module for tracking errors related to AI interactions

Tracks the error using AIErrorTracker, passing relevant context data

Creates a user-friendly error message based on the tracked error data

Prepares the display content for rendering an error message in the chat interface

I've created an example `AIErrorTracker` class, located in the error-tracking.js file within the lib folder, which encapsulates all the supported error types you may encounter when dealing with an AI provider, along with corresponding error messages that can be displayed to the user. This approach to error handling allows easier management of errors in a consistent manner. You can add more checks by extending the `ErrorType` and `ERROR_PATTERNS` constants within the `AIErrorTracker` class to cover additional error types.

Another benefit of this approach is that for certain errors, like network errors, you might want to prompt users to check their internet connection or try again later. Conversely, if an authentication error arises, you could display a message asking users to log in again or verify their credentials. Here is a snippet of code that adds a retry server action in the display message as an example: